06.3: MongoDB Config

UNIVERSITY OF MARYLAND

UMD DATA605 - Big Data Systems

06.3: MongoDB Config

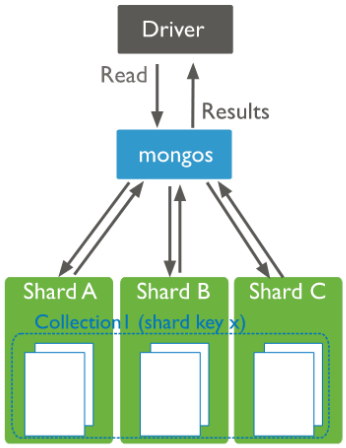
- **Instructor**: Dr. GP Saggese, gsaggese@umd.edu
- @References@:
  - All concepts in slides
  - MongoDB tutorial
  - Web
    - https://www.mongodb.com/
    - Official docs
    - pymongo
  - Seven Databases in Seven Weeks, 2e

1 / 12

2 / 12: MongoDB Processes and Configuration

MongoDB Processes and Configuration

- `mongod`: database instance (server process)
- `mongosh`: interactive shell (client)
  - JavaScript environment for MongoDB
- `mongos`: database router
  - Process requests
  - Decide which `mongod` instances receive the query (sharding/partitioning)
  - Collate results
  - Send result to client
- You should have:
  - One `mongos` (router) for the system regardless of `mongod` count; or
  - One local `mongos` per client to minimize network latency



2 / 12

- **`mongod`: Database Instance (Server Process)**
  - This is the core server process for MongoDB. It handles data storage, retrieval, and management. Think of it as the engine that powers the database.
- **`mongosh`: Interactive Shell (Client)**
  - This is a command-line interface for interacting with MongoDB. It provides a JavaScript environment, allowing users to execute queries and manage the database directly.
- **`mongos`: Database Router**
  - Acts as a traffic controller for database requests. It processes incoming queries and determines which `mongod` instances should handle them, especially in a sharded setup.
  - It collates results from different shards and sends the final result back to the client.
- **Configuration Recommendations**
  - You can have a single `mongos` for the entire system, which simplifies management.
  - Alternatively, having a local `mongos` per client can reduce network latency, improving performance.

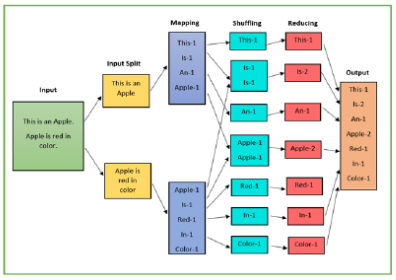
The diagram illustrates how `mongos` interacts with different shards, directing queries and collating results to provide seamless data access across a distributed database system.

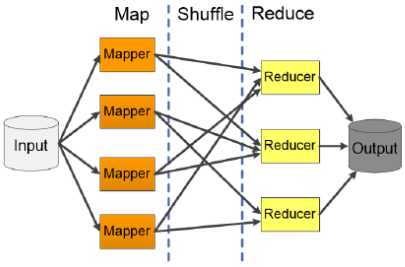
3 / 12: MapReduce Functionality

MapReduce Functionality

- Perform map-reduce computation on (key, value) pairs
- Provide map function, reduction function, and result set name

```
db.collection.mapReduce(
  <map_function>,
  <reduce_function>,
  {
    out: <collection>,
    query: <document>,
    sort: <document>,
    limit: <number>,
    finalize: <function>,
    scope: <document>,
    jsMode: <boolean>,
    verbose: <boolean>
  })
```





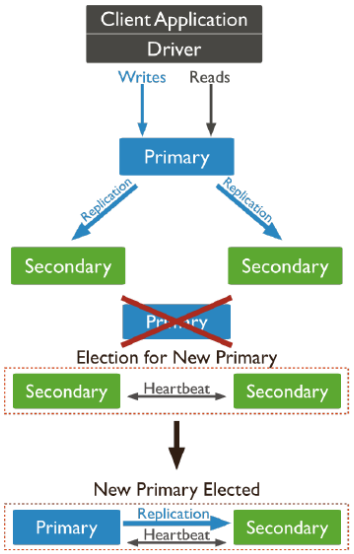
3 / 12

- **Perform map-reduce computation on (key, value) pairs**
  - MapReduce is a programming model used for processing large data sets with a distributed algorithm on a cluster.
  - It involves two main functions: *map* and *reduce*. The map function processes input data and produces key-value pairs, while the reduce function aggregates these pairs.
- **Provide map function, reduction function, and result set name**
  - The code snippet shows how to implement MapReduce in a database context using a function call.
  - **`<map_function>`**: Defines how to transform input data into key-value pairs.
  - **`<reduce_function>`**: Specifies how to combine values associated with the same key.
  - **`out: <collection>`**: Determines where the result will be stored.
  - Additional options like **`query`**, **`sort`**, and **`limit`** allow for more control over the data processing.
- **Illustrations**
  - The first image illustrates the flow of data through the MapReduce process: input is split, mapped, shuffled, and reduced to produce the output.
  - The second image shows a more abstract representation of the MapReduce architecture, highlighting the separation of the map, shuffle, and reduce phases.

4 / 12: Data Replication

Data Replication

- **Data replication** ensure:
  - Redundancy
  - Backup
  - Automatic failover
- Replication occurs through groups of servers known as **replica sets**
  - **Primary set**: servers for direct updates
  - **Secondary set**: servers for data duplication
  - Properties of secondary set:
    - Secondary-only, hidden, delayed, arbiters, non-voting
- If primary fails, secondary sets "vote" to elect new primary



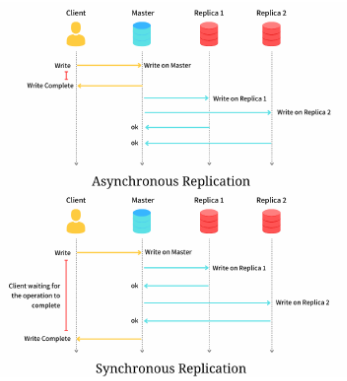
4 / 12

- **Data Replication** ensures several critical aspects of data management:
  - **Redundancy**: This means having multiple copies of data to prevent data loss.
  - **Backup**: Replication acts as a backup, safeguarding data against failures.
  - **Automatic Failover**: If a primary server fails, the system automatically switches to a backup without manual intervention.
- **Replication** is managed through groups of servers called *replica sets*:
  - **Primary Set**: This is where data updates occur directly. It handles all write operations.
  - **Secondary Set**: These servers duplicate data from the primary set. They can have various properties:
    - *Secondary-only*: Only stores replicated data.
    - *Hidden*: Not visible to client applications.
    - *Delayed*: Updates occur after a set delay.
    - *Arbiters*: Participate in elections but do not store data.
    - *Non-voting*: Do not participate in elections.
- In case the **primary server fails**, the secondary sets engage in a voting process to elect a new primary server, ensuring continuous availability and reliability.

5 / 12: Sync vs Async Replication

Sync vs Async Replication

- **Synchronous replication**: updates propagate to replicas in a single transaction
- Implementations
  - 2-Phase Commit (2PC)
  - Paxos
  - Complex and expensive
- **Asynchronous replication**
  - Primary node propagates updates to replicas
  - Transaction completes before replicas update (even if failures occur)
  - Quick commits at consistency cost

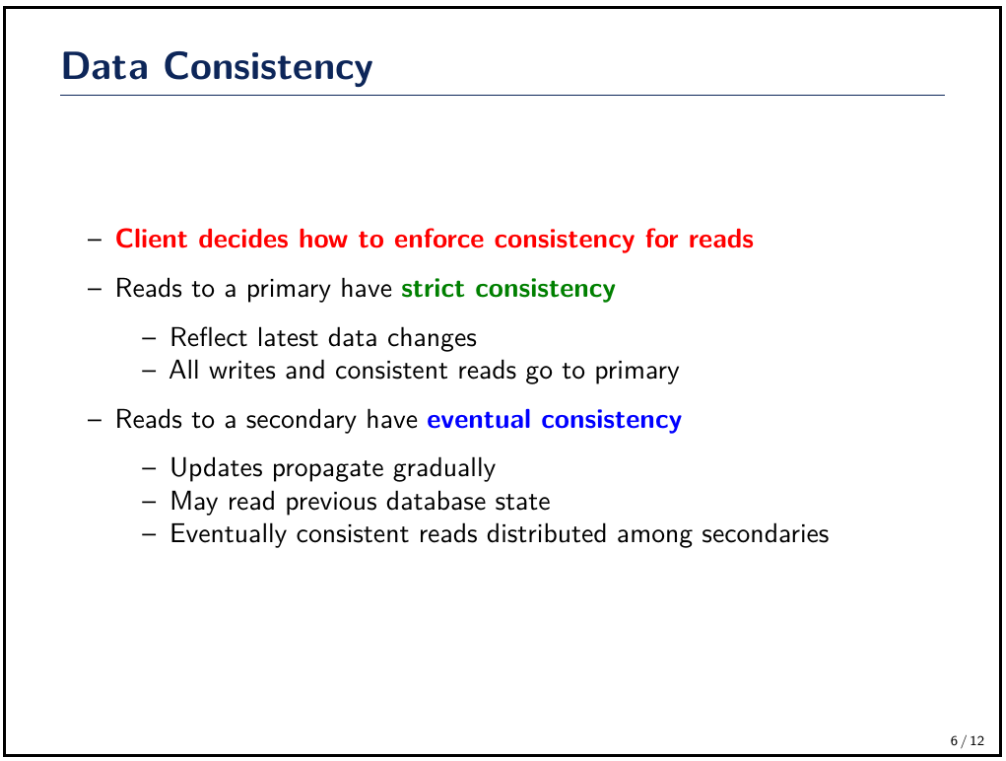


5 / 12

- **Synchronous Replication**: This method ensures that updates are propagated to all replicas within a single transaction. This means that the client must wait for confirmation from all replicas before the transaction is considered complete. This approach is reliable but can be complex and costly to implement.
- **Implementations**:
  - *2-Phase Commit (2PC)*: A protocol that ensures all nodes in a distributed system agree on a transaction before it is committed.
  - *Paxos*: A consensus algorithm that helps achieve agreement among distributed systems.
  - These methods are often complex and require significant resources.
- **Asynchronous Replication**: In this approach, the primary node sends updates to replicas, but the transaction is considered complete before the replicas are updated. This allows for faster transaction commits but may lead to temporary inconsistencies if failures occur.
  - The primary advantage is quick commits, which can improve performance.
  - However, this comes at the cost of consistency, as replicas may not always have the latest data immediately.

The diagrams illustrate the differences in communication flow between synchronous and asynchronous replication, highlighting the waiting period in synchronous replication and the immediate transaction completion in asynchronous replication.

6 / 12: Data Consistency



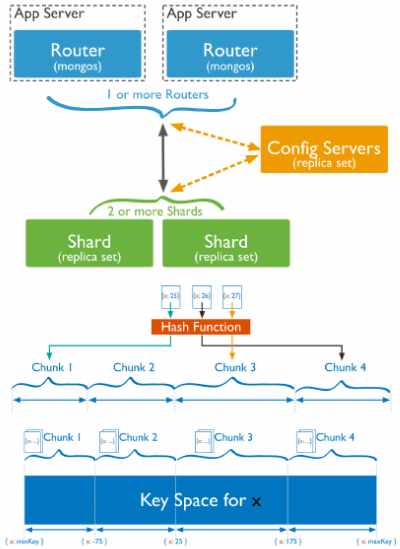
- **Data Consistency**
  - *Data consistency* is about ensuring that data remains accurate and reliable across different parts of a system. It's crucial for maintaining trust in the data being used.
- **@Client decides how to enforce consistency for reads@**
  - The *client* refers to the application or user accessing the database. They have the flexibility to choose how strictly they want the data to be consistent when they read it.
  - This choice impacts how up-to-date the data is when it's retrieved.
- **Reads to a primary have @strict consistency@**
  - **Reflect latest data changes**
    - When you read from the *primary* database, you get the most recent data. This means any changes made are immediately visible.
  - **All writes and consistent reads go to primary**
    - All data updates (writes) and reads that require the most current data are directed to the primary. This ensures that the data is always up-to-date.
- **Reads to a secondary have @eventual consistency@**
  - **Updates propagate gradually**
    - Changes made to the data take some time to reach the *secondary* databases. This means there might be a delay before the secondary reflects the latest updates.
  - **May read previous database state**
    - When reading from a secondary, you might get an older version of the data because updates haven't reached it yet.
  - **Eventually consistent reads distributed among secondaries**
    - Over time, all secondary databases will catch up with the primary, achieving consistency. Reads are spread across these secondaries to balance the load and improve

performance.

7 / 12: MongoDB: Sharding

MongoDB: Sharding

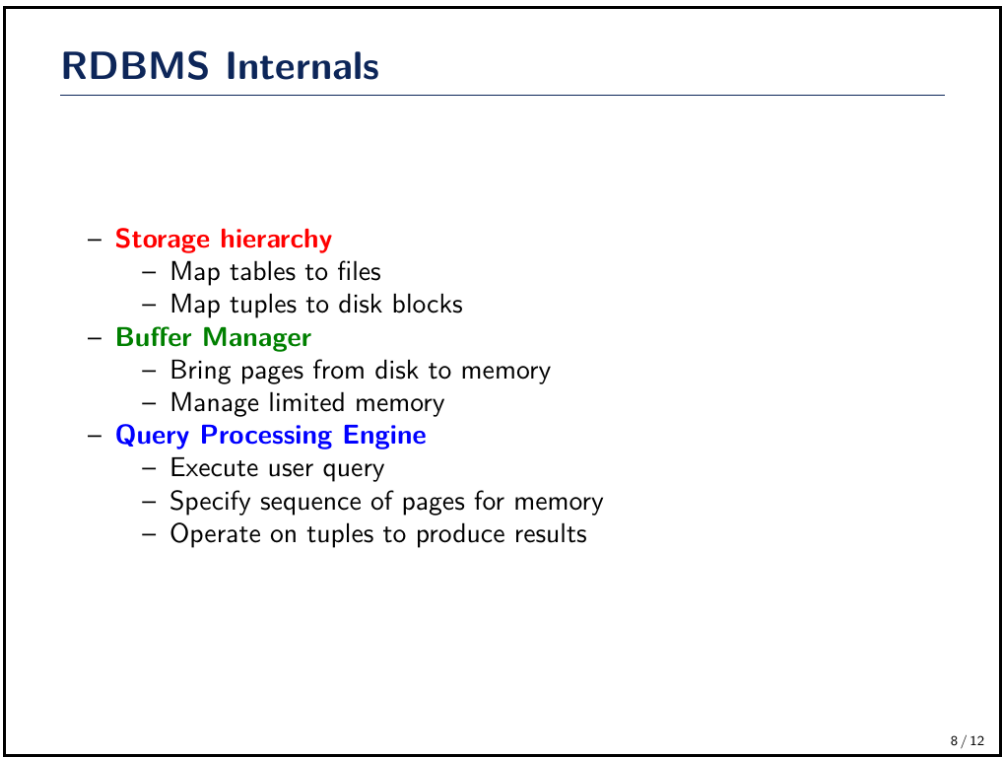
- **Shard** = subset of data
  - Split collection based on shard key
  - Distribute data based on shard key or intervals [a, b)
- **Sharding** = method for distributing data across machines
- **Horizontal scaling** achieved through sharding
  - Divide data and workload over servers
  - Complexity in infrastructure and maintenance
- `mongos` acts as query router interfacing clients and sharded cluster
  - Deploy each shard as a replica set
  - Config servers store metadata and configuration settings for



7 / 12

- **Shard**: A shard is essentially a subset of data. In MongoDB, data is split based on a *shard key*, which is a specific field or set of fields. This allows the data to be distributed across different shards based on specific intervals or ranges, such as [a, b).
- **Sharding**: This is the method used to distribute data across multiple machines. It helps in managing large datasets by breaking them into smaller, more manageable pieces, which can be stored on different servers.
- **Horizontal scaling**: Sharding enables horizontal scaling, which means adding more servers to handle increased load. This divides both data and workload across multiple servers, though it introduces complexity in terms of infrastructure and maintenance.
- `mongos`: This component acts as a query router. It interfaces between clients and the sharded cluster, directing queries to the appropriate shards. Each shard is deployed as a replica set, ensuring data redundancy and availability. Config servers are crucial as they store metadata and configuration settings for the cluster, ensuring everything runs smoothly.

8 / 12: RDBMS Internals



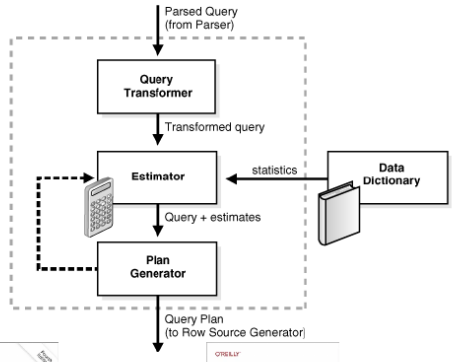
- **RDBMS Internals**
- *Storage hierarchy*
  - **Map tables to files**: In a relational database management system (RDBMS), tables are stored as files on disk. This means that each table, which is a collection of rows and columns, is represented by a file or a set of files. This mapping is crucial because it determines how data is physically stored and accessed.
  - **Map tuples to disk blocks**: A tuple is essentially a row in a table. These tuples are stored in disk blocks, which are fixed-size units of storage. Mapping tuples to disk blocks efficiently is important for quick data retrieval and storage.
- *Buffer Manager*
  - **Bring pages from disk to memory**: The buffer manager is responsible for loading data from disk into the main memory. This is necessary because accessing data in memory is much faster than accessing it on disk.
  - **Manage limited memory**: Memory is a limited resource, so the buffer manager must decide which data to keep in memory and which to evict. This involves strategies to optimize the use of available memory while minimizing the need to access slower disk storage.
- *Query Processing Engine*
  - **Execute user query**: This component takes the SQL queries provided by users and executes them. It translates the high-level query into a series of operations that the database can perform.

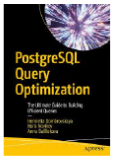
- **Specify sequence of pages for memory**: The engine determines the order in which data pages are accessed and processed in memory. This sequence is crucial for efficient query execution.
- **Operate on tuples to produce results**: The engine processes the data (tuples) according to the query's requirements, such as filtering, joining, or aggregating, to produce the desired results for the user.

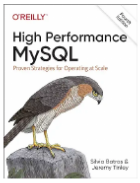
9 / 12: Query Optimizer

Query Optimizer

- **RDBMS: query optimizer is static**
  - Assign cost to each query plan
  - Estimate cost params (e.g., time to access data)
  - Search for best query
  - At least traditional RDBMS







9 / 12

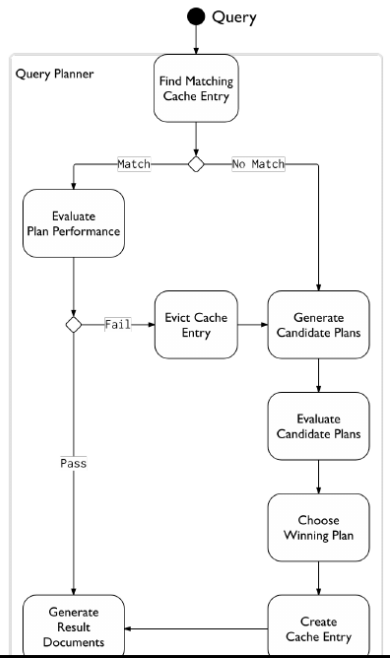
- **Query Optimizer in RDBMS**:
  - In a relational database management system (RDBMS), the *query optimizer* is a crucial component that operates in a static manner.
  - It assigns a *cost* to each potential query plan. This cost estimation helps in determining the efficiency of executing a query.
  - The optimizer estimates various parameters, such as the *time required to access data*. These estimates are based on statistics and data distribution.
  - The primary goal is to *search for the best query plan* that minimizes resource usage and execution time.
  - This process is a standard feature in traditional RDBMS systems, ensuring efficient data retrieval.
- **Diagram Explanation**:
  - The diagram illustrates the components involved in query optimization.
  - A *Query Transformer* modifies the parsed query to improve efficiency.
  - The *Estimator* uses statistics from the *Data Dictionary* to predict the cost of different query plans.
  - The *Plan Generator* creates the optimal query plan based on these estimates.

- **Books on Query Optimization**:
  - These books provide insights and strategies for optimizing queries in different database systems like PostgreSQL, MySQL, and SQL Server.
  - They are valuable resources for understanding advanced techniques and best practices in query optimization and performance tuning.

10 / 12: Query Optimizer

Query Optimizer

- **MongoDB: query optimizer is dynamic**
  - Try different query plans, learn which perform well
  - Query plan space is small, no joins
  - Testing new plans
    - Execute multiple plans in parallel
    - Terminate others when one finishes
  - Cache result
  - If a plan performs poorly, try different plans
    - E.g., data changed, query parameters differ

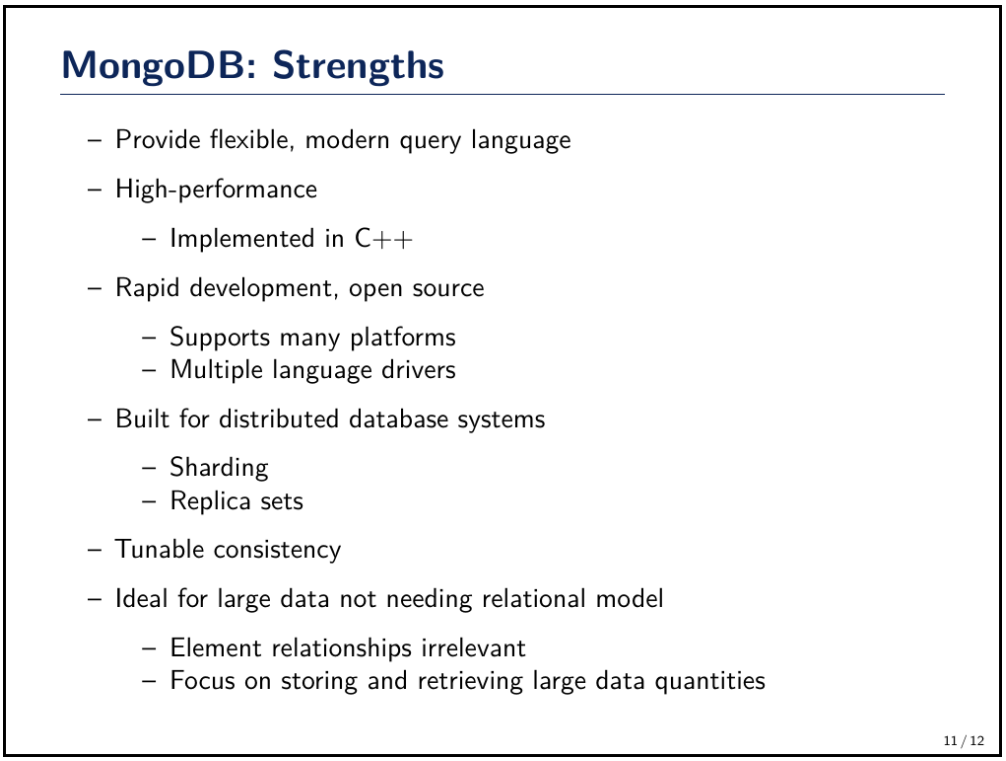


10 / 12

- **MongoDB's Query Optimizer**: MongoDB uses a dynamic query optimizer to improve query performance.
  - **Dynamic Plan Testing**: It tries different query plans to determine which performs best. This is crucial because the query plan space is relatively small, as MongoDB does not use joins.
  - **Parallel Execution**: Multiple query plans can be executed in parallel. Once one plan finishes, the others are terminated, ensuring efficiency.
  - **Caching**: Results from successful query plans are cached. This means if the same query is run again, MongoDB can quickly retrieve the results without recalculating.
  - **Adapting to Changes**: If a query plan starts performing poorly, perhaps due to changes in data or query parameters, MongoDB will try different plans to find a better one.
- **Flowchart Explanation**:
  - **Cache Matching**: The process begins by checking if there is a matching cache entry for the query.
  - **Plan Evaluation**: If a match is found, the plan's performance is evaluated. If it fails, the cache entry is evicted.
  - **Candidate Plans**: If no match is found, new candidate plans are generated and evalu-

ated.
- **Choosing a Plan**: The best-performing plan is chosen and a new cache entry is created.
- **Result Generation**: Finally, the chosen plan is used to generate the result documents.

11 / 12: MongoDB: Strengths



- **MongoDB: Strengths**
- **Provide flexible, modern query language**
  - MongoDB uses a query language that is designed to be flexible and easy to use. This means you can perform complex queries on your data without needing to write complicated code. It's similar to SQL but tailored for the document-oriented nature of MongoDB.
- **High-performance**
  - **Implemented in C++**
    - MongoDB is built using C++, a programming language known for its speed and efficiency. This contributes to MongoDB's ability to handle large volumes of data quickly and efficiently.
- **Rapid development, open source**
  - **Supports many platforms**
    - Being open source means MongoDB is free to use and can be modified by anyone. It also runs on various operating systems, making it versatile for different development environments.
  - **Multiple language drivers**

        * MongoDB provides drivers for many programming languages, allowing developers to integrate it easily into their applications, regardless of the language they are using.

- **Built for distributed database systems**
  - **Sharding**
    * Sharding is a method for distributing data across multiple servers. MongoDB uses sharding to ensure that large datasets can be spread out, improving performance and scalability.
  - **Replica sets**
    * Replica sets are a way to ensure data redundancy and high availability. MongoDB can automatically replicate data across different servers, so if one server fails, another can take over.

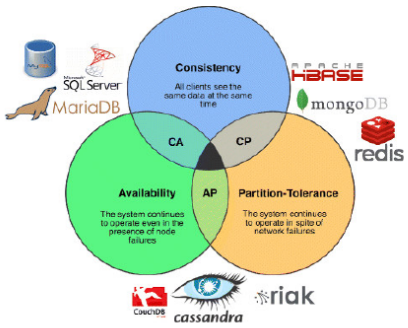
- **Tunable consistency**
  - MongoDB allows you to adjust the level of consistency according to your needs. This means you can choose between strong consistency, where data is always up-to-date, or eventual consistency, which can improve performance.

- **Ideal for large data not needing relational model**
  - **Element relationships irrelevant**
    * MongoDB is perfect for situations where the relationships between data elements are not important. This is because it doesn't use a traditional relational database model.
  - **Focus on storing and retrieving large data quantities**
    * The primary goal of MongoDB is to efficiently store and retrieve large amounts of data, making it a great choice for applications that handle big data.

12 / 12: MongoDB: Limitations

MongoDB: Limitations

- No referential integrity
  - Aka foreign key constraint
- Lack of transactions and joins
- High degree of denormalization
  - Update data in many places instead of one
- Lack of predefined schema is a double-edged sword
  - Have a data model in your application
  - Objects within a collection can be inconsistent in their fields
- CAP Theorem: targets consistency and partition tolerance, gives up on availability



12 / 12

- **No referential integrity**
  - *Referential integrity* refers to the enforcement of foreign key constraints, which ensure that relationships between tables remain consistent. MongoDB does not support this, meaning it doesn't automatically enforce relationships between collections. This can lead to potential data inconsistencies if not managed carefully by the application.
- **Lack of transactions and joins**
  - Traditional databases allow for transactions and joins, which help maintain data integrity and simplify complex queries. MongoDB, being a NoSQL database, traditionally lacked these features, although recent versions have introduced multi-document transactions. However, the absence of joins means that data often needs to be denormalized.
- **High degree of denormalization**
  - Denormalization involves storing data in multiple places to improve read performance. In MongoDB, this means that updates might need to be applied in several locations, increasing the risk of data inconsistency and complicating data management.
- **Lack of predefined schema is a double-edged sword**
  - MongoDB's flexibility allows for dynamic schemas, meaning documents in a collection can have different fields. While this provides flexibility, it can lead to inconsistencies and requires the application to enforce a data model.
- **CAP Theorem: targets consistency and partition tolerance, gives up on availability**
  - According to the CAP Theorem, a distributed database can only guarantee two out of three: Consistency, Availability, and Partition Tolerance. MongoDB prioritizes consistency and partition tolerance, which means it may sacrifice availability during network partitions. This trade-off is important to consider based on application needs.